The Diophantine equation $x^2 = n! \cdot n\#$: the complete solution set

(Lean 4 machine-checked claim of record)

Aurora0134*

October 6, 2026

Abstract

We prove that the Diophantine equation $x^2 = n! \cdot n\#$ (the factorial times the primorial) has, for $n \in \mathbb{N}$, exactly the solutions $n \in \{0, 1, 2, 3, 4, 5\}$, with $x = 1, 1, 2, 6, 12, 60$. This settles one of the two cases that Novaković (2026) recorded as believed-finite with the words "at the moment we were not able to give an argument" [3]. The result is formalized in Lean 4 against mathlib v4.34.0: every frozen statement compiles with no `sorry`, and `#print axioms` reports only {propext, `Quot.sound`, `Classical.choice`}. A novelty review (full query log shipped in this deposit) found no occupying record in the queried channels. This note is a priority claim of record, not a full paper; the Lean sources and the audit and novelty evidence ship inside this deposit.

1 Introduction

This note records a complete-solution-set theorem formalized in Lean 4, with the kernel as the sole adjudicator at every step: the statements were frozen by hash before proving, and a separate final audit re-compared them against the frozen snapshot, scanned the whole proof file for `sorry`/`admit` including comments, and re-printed the axiom dependencies [1, 2]. The object is the Diophantine equation of the title, one of the concretely open cases listed by Novaković [3]. The note is a priority claim of record, deposited for timestamping; a narrative paper is prepared separately and will reference this deposit. No literature survey is included—that expectation belongs to arXiv moderation and does not apply to a Zenodo deposit.

*Contact: via the GitHub account Aurora0134.

2 Statement

We write $n!$ for the factorial and $n\# = \prod_{p \le n,\ p \text{ prime}} p$ for the primorial; $\nu_p(m)$ denotes the exponent of the prime p in m. In the formalization, `primorial` is the top-level mathlib declaration and $\nu_p(m)$ is `Nat.factorization m p`. Division in Lemma 4 is natural-number division, so the interval reads $\lfloor n/3 \rfloor < p \le \lfloor n/2 \rfloor$; for prime existence this coincides with the real reading, since p is an integer.

Theorem 1 (complete solution set, iff form)**.** *For every $n \in \mathbb{N}$, $n! \cdot n\#$ is a perfect square if and only if $n \le 5$.*

Theorem 2 (existence-witness form)**.** *For every $n \in \mathbb{N}$, the equation $x^2 = n! \cdot n\#$ has a solution $x \in \mathbb{N}$ if and only if $n \in \{0, 1, 2, 3, 4, 5\}$.*

Lemma 3 (valuation criterion)**.** *For $n \ge 1$: $n! \cdot n\#$ is a perfect square if and only if $\nu_p(n!)$ is odd for every prime $p \le n$.*

Lemma 4 (interval prime lemma)**.** *For every integer $n \ge 22$ there exists a prime p with $n/3 < p \le n/2$.*

Theorem 5 (solution witnesses)**.** $1^2 = 0! \cdot 0\#$, $1^2 = 1! \cdot 1\#$, $2^2 = 2! \cdot 2\#$, $6^2 = 3! \cdot 3\#$, $12^2 = 4! \cdot 4\#$, *and* $60^2 = 5! \cdot 5\#$.

Three auxiliary factorization statements belong to the same frozen set and carry the same grade (verbatim in Listing 1): $\nu_p(n\#) = 1$ for primes $p \le n$, $\nu_p(n\#) = 0$ for primes $p \nleq n$, and $\nu_p(n! \cdot n\#) = \nu_p(n!) + 1$ for primes $p \le n$.

Listing 1: Frozen statement snapshot (Lean 4): the complete frozen set, copied from the `lean` fence of the snapshot shipped as `proofs/nfact-primorial-square.statement.txt` (a comment-cleaned edition: docstrings translated to English for publication; the statement text is byte-identical to the frozen snapshot of record, sha256 `0b13d08d`; correspondence record `audit/comment-cleanup-diff.md`).

```
/-- **Main theorem (complete solution set, iff form)**:
    `n! * n#` is a perfect square ⟺ `n ≤ 5` (for `n : ℕ`).
    Over ℕ, `n ≤ 5 ⟺ n ∈ {0,1,2,3,4,5}`, so this is the main
    statement "the solutions are exactly n ∈ {0,...,5}". -/
theorem isSquare_factorial_mul_primorial_iff (n : ℕ) :
    IsSquare (n ! * primorial n) ↔ n ≤ 5

/-- **Main theorem (existence-witness form, delivered alongside
    the iff form)**:
```

```
    the Diophantine equation `x^2 = n! * n#` has a solution ⟺ `n
    ∈ {0,1,2,3,4,5}`. -/
theorem exists_sq_factorial_mul_primorial_iff (n : ℕ) :
    (∃ x : ℕ, x ^ 2 = n ! * primorial n) ↔ n ∈ ({0, 1, 2, 3, 4,
    5} : Finset ℕ)

/-- **Lemma A (equivalent valuation criterion)**: for `n ≥ 1`, `n
    ! * n#` is a perfect square ⟺
    `ν_p(n!)` is odd for every prime `p ≤ n`. -/
theorem isSquare_iff_all_factorization_odd (n : ℕ) (hn : 1 ≤ n) :
    IsSquare (n ! * primorial n) ↔
      ∀ p : ℕ, p.Prime → p ≤ n → Odd ((n !).factorization p)

/-- **Lemma B (interval prime lemma)**: for every integer `n ≥
    22` there exists a prime `p` with `n/3 < p ≤ n/2`. -/
theorem exists_prime_between_div_three_and_div_two (n : ℕ) (hn :
    22 ≤ n) :
    ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2

/-- **Witness layer (explicit square roots for the ⟸ direction
    of the main theorem)**: `x^2 = n! * n#` at each of n = 0..5,
    with x = 1, 1, 2, 6, 12, 60. -/
theorem witnesses_zero_to_five :
    1 ^ 2 = 0 ! * primorial 0 ∧
    1 ^ 2 = 1 ! * primorial 1 ∧
    2 ^ 2 = 2 ! * primorial 2 ∧
    6 ^ 2 = 3 ! * primorial 3 ∧
    12 ^ 2 = 4 ! * primorial 4 ∧
    60 ^ 2 = 5 ! * primorial 5

/-- Auxiliary statement: `ν_p(n#) = 1` (primes `p ≤ n`). -/
theorem primorial_factorization_eq_one {p n : ℕ} (hp : p.Prime) (
    hpn : p ≤ n) :
    (primorial n).factorization p = 1

/-- Auxiliary statement: `ν_p(n#) = 0` (primes `p ≰ n`). -/
theorem primorial_factorization_eq_zero {p n : ℕ} (hp : p.Prime) (
    hpn : ¬ p ≤ n) :
    (primorial n).factorization p = 0

/-- Auxiliary statement (the multiplicative skeleton of the
    criterion): `ν_p(n! * n#) = ν_p(n!) + 1` (primes `p ≤ n`). -/
theorem factorization_factorial_mul_primorial {p n : ℕ} (hp : p.
    Prime) (hpn : p ≤ n) :
    (n ! * primorial n).factorization p = (n !).factorization p +
```

```
1
```

3 Proof

The proof assembles four components. (1) *The valuation criterion* (Lemma 3): $n\#$ contributes each prime $p \le n$ exactly once, so $n! \cdot n\#$ is a square if and only if every $\nu_p(n!)$ is odd; both directions go through the library characterization of squares by even factorizations. (2) *Exclusion of large n*: for $n \ge 22$, Lemma 4 yields a prime p with $n/3 < p \le n/2$; then $2p \le n < 3p$ and $p^2 > n$, so Legendre's formula gives $\nu_p(n!) = \lfloor n/p \rfloor + \lfloor n/p^2 \rfloor + \cdots = 2$, an even number, contradicting the all-odd criterion. Lemma 4 itself is proved in two regimes. For $22 \le n \le 999999$, a certificate chain of twenty-nine primes (Listing 2), in which each prime q covers $2q \le n \le 3q - 1$ and consecutive certificates overlap, closes the range by `norm_num`/`omega`. For $n \ge 10^6$, an Erdős-style factorial-ratio argument (Listing 3): with $\mathrm{tailR}(n) = (n/2)!\,(n/6)!\,/\,((n/3)!\,(n/4)!\,(n/12)!)$ one has $18^{\lfloor n/12 \rfloor} < \mathrm{tailR}(n) \cdot (4\lfloor n/12 \rfloor + 3)^6$, while the absence of primes in $(n/3, n/2]$ would force $\mathrm{tailR}(n)$ below essentially $16^{\lfloor n/12 \rfloor}$ times lower-order factors; the decisive inequality $18 > 16$ is closed by a concavity two-point estimate. The segment $10^6 \le n < 1.2 \cdot 10^6$ is closed by a single certificate hop with $q = 400009$. (3) *Small n*: witnesses for $n = 0, \ldots, 5$ (Theorem 5) and, for $6 \le n \le 21$, sixteen explicit factorizations each exhibiting a prime with odd exponent (Listing 4). (4) *Assembly*: the two main theorems follow from the criterion, the exclusion, and the witnesses.

For honesty of the record: the route originally envisaged for Lemma 4 (splicing mathlib's Bertrand theorem across two dyadic sub-intervals) was strictly refuted during the task—a countermodel (the set of all primes with 13 removed) satisfies the Bertrand property yet leaves the window $(11, 16]$ empty, so no black-box argument at ratio 2 can close a window of ratio $3/2$—and the shipped route is the certificate chain plus factorial-ratio tail described above. The complete proof file (including the twenty-nine prime certificates and the twenty-eight continuity certificates, which are one-line `norm_num`/`omega` facts) ships in `proofs/`; the blocks below are verbatim excerpts, each a contiguous line range of that file. The shipped file is the comment-cleaned edition; its code is byte-identical to the frozen development (record: `audit/comment-cleanup-diff.md`).

Listing 2: Excerpt of the proof file (lines 96–168 of `final-nfact-primorial-square.lean`, which ships in full in `proofs/`):

the certificate-chain mechanism for Lemma 4 on $22 \le n \le 999999$ — the two boundary certificates, the single-hop lemma `lemmaB_hop`, and the twenty-nine-hop decision tree.

```
theorem chain_start : 22 = 2 * 11 := by omega
theorem chain_end : 999999 ≤ 3 * 409609 - 1 := by omega

/-- Certificate-chain single hop: for prime `q` and `n ∈ [2q, 3q - 1]`,
    `q` itself witnesses `n/3 < q ≤ n/2`.
    Note: extract `hq.two_le` first, otherwise `omega` gets stuck at the
    `q = 0` counterexample under saturating subtraction on ℕ. -/
theorem lemmaB_hop {q : ℕ} (hq : q.Prime) {n : ℕ} (h1 : 2 * q ≤ n) (h2 :
     n ≤ 3 * q - 1) :
    ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2 :=
  have hq2 : 2 ≤ q := hq.two_le
  ⟨q, hq, by omega, by omega⟩

/-- **Lemma B, finite segment (certificate-chain decision tree)**: for
    every `n ∈ [22, 999999]`,
    `∃ p, p.Prime ∧ n/3 < p ∧ p ≤ n/2`.
    Joins the tail theorem (which starts at n = 1000000) by a case split
    on `n < 1000000`. -/
theorem lemmaB_upto_999999 (n : ℕ) (h1 : 22 ≤ n) (h2 : n ≤ 999999) :
    ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2 := by
  rcases lt_or_ge n 33 with h | h
  · exact lemmaB_hop (q := 11) prime_cert_11 (by omega) (by omega)
  · rcases lt_or_ge n 39 with h | h
    · exact lemmaB_hop (q := 13) prime_cert_13 (by omega) (by omega)
    · rcases lt_or_ge n 57 with h | h
      · exact lemmaB_hop (q := 19) prime_cert_19 (by omega) (by omega)
      · rcases lt_or_ge n 69 with h | h
        · exact lemmaB_hop (q := 23) prime_cert_23 (by omega) (by omega)
        · rcases lt_or_ge n 93 with h | h
          · exact lemmaB_hop (q := 31) prime_cert_31 (by omega) (by omega)
          · rcases lt_or_ge n 129 with h | h
            · exact lemmaB_hop (q := 43) prime_cert_43 (by omega) (by
    omega)
            · rcases lt_or_ge n 183 with h | h
              · exact lemmaB_hop (q := 61) prime_cert_61 (by omega) (by
    omega)
              · rcases lt_or_ge n 267 with h | h
                · exact lemmaB_hop (q := 89) prime_cert_89 (by omega) (by
    omega)
                · rcases lt_or_ge n 393 with h | h
                  · exact lemmaB_hop (q := 131) prime_cert_131 (by omega)
    (by omega)
                  · rcases lt_or_ge n 579 with h | h
                    · exact lemmaB_hop (q := 193) prime_cert_193 (by omega
    ) (by omega)
```

```
                · rcases lt_or_ge n 849 with h | h
                  · exact lemmaB_hop (q := 283) prime_cert_283 (by omega) (by omega)
                · rcases lt_or_ge n 1263 with h | h
                  · exact lemmaB_hop (q := 421) prime_cert_421 (by omega) (by omega)
                · rcases lt_or_ge n 1893 with h | h
                  · exact lemmaB_hop (q := 631) prime_cert_631 (by omega) (by omega)
                · rcases lt_or_ge n 2823 with h | h
                  · exact lemmaB_hop (q := 941) prime_cert_941 (by omega) (by omega)
                · rcases lt_or_ge n 4227 with h | h
                  · exact lemmaB_hop (q := 1409) prime_cert_1409 (by omega) (by omega)
                · rcases lt_or_ge n 6339 with h | h
                  · exact lemmaB_hop (q := 2113) prime_cert_2113 (by omega) (by omega)
                · rcases lt_or_ge n 9507 with h | h
                  · exact lemmaB_hop (q := 3169) prime_cert_3169 (by omega) (by omega)
                · rcases lt_or_ge n 14253 with h | h
                  · exact lemmaB_hop (q := 4751) prime_cert_4751 (by omega) (by omega)
                · rcases lt_or_ge n 21363 with h | h
                  · exact lemmaB_hop (q := 7121) prime_cert_7121 (by omega) (by omega)
                · rcases lt_or_ge n 32001 with h | h
                  · exact lemmaB_hop (q := 10667) prime_cert_10667 (by omega) (by omega)
                · rcases lt_or_ge n 47973 with h | h
                  · exact lemmaB_hop (q := 15991) prime_cert_15991 (by omega) (by omega)
                · rcases lt_or_ge n 71943 with h | h
                  · exact lemmaB_hop (q := 23981) prime_cert_23981 (by omega) (by omega)
                · rcases lt_or_ge n 107907 with h | h
                  · exact lemmaB_hop (q := 35969) prime_cert_35969 (by omega) (by omega)
                · rcases lt_or_ge n 161853 with h | h
                  · exact lemmaB_hop (q := 53951) prime_cert_53951 (by omega) (by omega)
                · rcases lt_or_ge n 242769 with h | h
                  · exact lemmaB_hop (q :=
```

```
     80923) prime_cert_80923 (by omega) (by omega)
67                                                  · rcases lt_or_ge n
    364137 with h | h
68                                                    · exact lemmaB_hop (q
    := 121379) prime_cert_121379 (by omega) (by omega)
69                                                  · rcases lt_or_ge n
    546177 with h | h
70                                                    · exact lemmaB_hop (
    q := 182059) prime_cert_182059 (by omega) (by omega)
71                                                  · rcases lt_or_ge n
    819249 with h | h
72                                                    · exact lemmaB_hop
     (q := 273083) prime_cert_273083 (by omega) (by omega)
73                                                    · exact lemmaB_hop
     (q := 409609) prime_cert_409609 (by omega) (by omega)
```

Listing 3: Excerpt of the proof file (lines 735–767): the tail theorem for $n \geq 10^6$ (`lemmaB_tail_1000000`), closing Lemma 4 on the infinite regime via the factorial-ratio estimate.

```
1  theorem lemmaB_tail_1000000 (n : ℕ) (hn : 1000000 ≤ n) :
2      ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2 := by
3    rcases lt_or_ge n 1200000 with h | h
4    · exact lemmaB_hop (q := 400009) (by norm_num) (by omega) (by omega)
5    · by_contra hc
6      push_neg at hc
7      have hc' : ∀ p : ℕ, p.Prime → n / 3 < p → ¬ p ≤ n / 2 := by
8        intro p hp1 hp2 hle
9        have hcc := hc p hp1 hp2
10       omega
11     have hub := tailR_le_of_no_prime n (by omega) hc'
12     have hlb := tailR_lower_strict n hn
13     have hmain := tail_main_inequality_nat (n/12) (by omega)
14     have h2 : n < (n/12 + 1) * 12 := (Nat.div_lt_iff_lt_mul (by norm_num
       : (0:ℕ) < 12)).mp (Nat.lt_succ_self _)
15     have e1 : (n/2)^(Nat.sqrt (n/2)) ≤ (6*(n/12)+5)^(Nat.sqrt (6*(n/12)
       +5)) := by
16       have hbase : n/2 ≤ 6*(n/12)+5 := by omega
17       have hs : Nat.sqrt (n/2) ≤ Nat.sqrt (6*(n/12)+5) := Nat.
       sqrt_le_sqrt hbase
18       exact (Nat.pow_le_pow_right (by omega : 0 < n/2) hs).trans (Nat.
       pow_le_pow_left hbase _)
19     have e2 : 4^(n/6) ≤ 4 * 16^(n/12) := by
20       have h4 : n/6 ≤ 2*(n/12) + 1 := by omega
21       calc 4^(n/6) ≤ 4^(2*(n/12)+1) := Nat.pow_le_pow_right (by norm_num)
         h4
22         _ = 4 * 16^(n/12) := by
23           rw [pow_succ, pow_mul]
```

```
          norm_num
          ring
    have key : 18^(n/12) < 18^(n/12) := by
      calc 18^(n/12) < tailR n * (4*(n/12)+3)^6 := hlb
        _ ≤ ((n/2)^(Nat.sqrt (n/2)) * 4^(n/6)) * (4*(n/12)+3)^6 := Nat.
    mul_le_mul_right _ hub
        _ ≤ ((6*(n/12)+5)^(Nat.sqrt (6*(n/12)+5)) * (4 * 16^(n/12))) *
    (4*(n/12)+3)^6 :=
            Nat.mul_le_mul_right _ (Nat.mul_le_mul e1 e2)
        _ = 4 * (4*(n/12)+3)^6 * (6*(n/12)+5)^(Nat.sqrt (6*(n/12)+5)) *
    16^(n/12) := by ring
        _ ≤ 18^(n/12) := hmain
    exact absurd key (lt_irrefl _)
```

Listing 4: Excerpt of the proof file (lines 771–1094): the main namespace `NFactPrimorialSquare` in full — the three auxiliary factorization lemmas, Lemma 3, the witness layer, the sixteen non-square witnesses, Lemma 4 assembled from the chain and the tail, and the two main theorems.

```
namespace NFactPrimorialSquare

/-- Auxiliary statement: `ν_p(n#) = 1` for primes `p ≤ n`. -/
theorem primorial_factorization_eq_one {p n : ℕ} (hp : p.Prime) (hpn : p
    ≤ n) :
    (primorial n).factorization p = 1 :=
  factorization_eq_one_of_squarefree (squarefree_primorial n) hp
    ((Nat.Prime.dvd_primorial_iff hp).mpr hpn)

/-- Auxiliary statement: `ν_p(n#) = 0` for primes `p ≰ n`. -/
theorem primorial_factorization_eq_zero {p n : ℕ} (hp : p.Prime) (hpn : ¬
     p ≤ n) :
    (primorial n).factorization p = 0 :=
  Nat.factorization_eq_zero_of_not_dvd
    (fun h => hpn ((Nat.Prime.dvd_primorial_iff hp).mp h))

/-- Auxiliary statement (the multiplicative skeleton of the criterion): `
    ν_p(n! * n#) = ν_p(n!) + 1` for primes `p ≤ n`. -/
theorem factorization_factorial_mul_primorial {p n : ℕ} (hp : p.Prime) (
    hpn : p ≤ n) :
    (n ! * primorial n).factorization p = (n !).factorization p + 1 := by
  rw [Nat.factorization_mul (Nat.factorial_ne_zero n) (primorial_ne_zero
    n),
      Finsupp.add_apply, primorial_factorization_eq_one hp hpn]

/-- **Lemma A (equivalent valuation criterion)**: for `n ≥ 1`, `n! * n#`
    is a perfect square ⟺
    `ν_p(n!)` is odd for every prime `p ≤ n`.
```

```
    (`Nat.factorization m p` is ν_p(m): the exponent of the prime `p` in
    `m`.) -/
theorem isSquare_iff_all_factorization_odd (n : ℕ) (hn : 1 ≤ n) :
    IsSquare (n ! * primorial n) ↔
      ∀ p : ℕ, p.Prime → p ≤ n → Odd ((n !).factorization p) := by
  constructor
  · intro h p hp hpn
    have hEven : Even ((n ! * primorial n).factorization p) :=
      Nat.isSquare_iff_even_factorization.mp h p hp
    rw [factorization_factorial_mul_primorial hp hpn] at hEven
    obtain ⟨m, hm⟩ := hEven
    cases m with
    | zero => simp at hm
    | succ k => exact ⟨k, by omega⟩
  · intro h
    apply Nat.isSquare_iff_even_factorization.mpr
    intro p hp
    by_cases hpn : p ≤ n
    · rw [factorization_factorial_mul_primorial hp hpn]
      exact (h p hp hpn).add_one
    · rw [Nat.factorization_mul (Nat.factorial_ne_zero n) (
    primorial_ne_zero n),
        Finsupp.add_apply,
        Nat.factorization_eq_zero_of_not_dvd
          (fun hd => hpn ((Nat.Prime.dvd_factorial hp).mp hd)),
        primorial_factorization_eq_zero hp hpn]
      exact ⟨0, rfl⟩

/-- **Witness layer (explicit square roots for the ⟸ direction of the
    main theorem)**: `x^2 = n! * n#`
    at each of n = 0..5, with x = 1, 1, 2, 6, 12, 60 (the pointwise
    correspondence of the main statement). -/
theorem witnesses_zero_to_five :
    1 ^ 2 = 0 ! * primorial 0 ∧
    1 ^ 2 = 1 ! * primorial 1 ∧
    2 ^ 2 = 2 ! * primorial 2 ∧
    6 ^ 2 = 3 ! * primorial 3 ∧
    12 ^ 2 = 4 ! * primorial 4 ∧
    60 ^ 2 = 5 ! * primorial 5 := by decide

/-- Bridge lemma (internal to the witness layer): an odd factorization at
     some prime implies non-square.
    `IsSquare` gives `m = r * r`, so `ν_p(m) = ν_p(r) + ν_p(r)` is even,
    contradicting the oddness assumption. -/
theorem not_isSquare_of_odd_factorization {m p e : ℕ} (hm : m ≠ 0)
    (h : m.factorization p = 2 * e + 1) : ¬ IsSquare m := by
  rintro ⟨r, hr⟩
  have hne : r ≠ 0 := by rintro rfl; rw [mul_zero] at hr; exact hm hr
  rw [hr, Nat.factorization_mul hne hne, Finsupp.add_apply] at h
```

```
  omega

/-- Non-square witness n = 6: `ν_2(6! * 6#) = 5` (odd); `6! * 6# = 2^5 *
    675`, and `2 ∤ 675`. -/
theorem nonsquare_at_6 : ¬ IsSquare (6 ! * primorial 6) := by
  have hM : (6 : ℕ) ! * primorial 6 = 2 ^ 5 * 675 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 2)
    (mul_ne_zero (Nat.factorial_ne_zero 6) (primorial_ne_zero 6))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^5) ≠ 0) (by
    norm_num : (675:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 675),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 7: `ν_2(7! * 7#) = 5` (odd); `7! * 7# = 2^5 *
    33075`, and `2 ∤ 33075`. -/
theorem nonsquare_at_7 : ¬ IsSquare (7 ! * primorial 7) := by
  have hM : (7 : ℕ) ! * primorial 7 = 2 ^ 5 * 33075 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 2)
    (mul_ne_zero (Nat.factorial_ne_zero 7) (primorial_ne_zero 7))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^5) ≠ 0) (by
    norm_num : (33075:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 33075),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 8: `ν_3(8! * 8#) = 3` (odd); `8! * 8# = 3^3 *
    313600`, and `3 ∤ 313600`. -/
theorem nonsquare_at_8 : ¬ IsSquare (8 ! * primorial 8) := by
  have hM : (8 : ℕ) ! * primorial 8 = 3 ^ 3 * 313600 := by decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 1)
    (mul_ne_zero (Nat.factorial_ne_zero 8) (primorial_ne_zero 8))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^3) ≠ 0) (by
    norm_num : (313600:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 313600),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 9: `ν_3(9! * 9#) = 5` (odd); `9! * 9# = 3^5 *
    313600`, and `3 ∤ 313600`. -/
theorem nonsquare_at_9 : ¬ IsSquare (9 ! * primorial 9) := by
  have hM : (9 : ℕ) ! * primorial 9 = 3 ^ 5 * 313600 := by decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 2)
```

```
    (mul_ne_zero (Nat.factorial_ne_zero 9) (primorial_ne_zero 9))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^5) ≠ 0) (by
    norm_num : (313600:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 313600),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 10: `ν_2(10! * 10#) = 9` (odd); `10! * 10# =
    2^9 * 1488375`, and `2 ∤ 1488375`. -/
theorem nonsquare_at_10 : ¬ IsSquare (10 ! * primorial 10) := by
  have hM : (10 : ℕ) ! * primorial 10 = 2 ^ 9 * 1488375 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 4)
    (mul_ne_zero (Nat.factorial_ne_zero 10) (primorial_ne_zero 10))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^9) ≠ 0) (by
    norm_num : (1488375:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 1488375),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 11: `ν_2(11! * 11#) = 9` (odd); `11! * 11# =
    2^9 * 180093375`, and `2 ∤ 180093375`. -/
theorem nonsquare_at_11 : ¬ IsSquare (11 ! * primorial 11) := by
  have hM : (11 : ℕ) ! * primorial 11 = 2 ^ 9 * 180093375 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 4)
    (mul_ne_zero (Nat.factorial_ne_zero 11) (primorial_ne_zero 11))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^9) ≠ 0) (by
    norm_num : (180093375:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 180093375),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 12: `ν_2(12! * 12#) = 11` (odd); `12! * 12# =
    2^11 * 540280125`, and `2 ∤ 540280125`. -/
theorem nonsquare_at_12 : ¬ IsSquare (12 ! * primorial 12) := by
  have hM : (12 : ℕ) ! * primorial 12 = 2 ^ 11 * 540280125 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 5)
    (mul_ne_zero (Nat.factorial_ne_zero 12) (primorial_ne_zero 12))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^11) ≠ 0) (by
    norm_num : (540280125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 540280125),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
```

```
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 13: `ν_2(13! * 13#) = 11` (odd); `13! * 13# =
    2^11 * 91307341125`, and `2 ∤ 91307341125`. -/
theorem nonsquare_at_13 : ¬ IsSquare (13 ! * primorial 13) := by
  have hM : (13 : ℕ) ! * primorial 13 = 2 ^ 11 * 91307341125 := by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 5)
    (mul_ne_zero (Nat.factorial_ne_zero 13) (primorial_ne_zero 13))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^11) ≠ 0) (by
    norm_num : (91307341125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 91307341125),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 14: `ν_5(14! * 14#) = 3` (odd); `14! * 14# =
    5^3 * 20943712677888`, and `5 ∤ 20943712677888`. -/
theorem nonsquare_at_14 : ¬ IsSquare (14 ! * primorial 14) := by
  have hM : (14 : ℕ) ! * primorial 14 = 5 ^ 3 * 20943712677888 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 5) (e := 1)
    (mul_ne_zero (Nat.factorial_ne_zero 14) (primorial_ne_zero 14))
  rw [hM, Nat.factorization_mul (by norm_num : ((5:ℕ)^3) ≠ 0) (by
    norm_num : (20943712677888:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (5:ℕ) ∣ 20943712677888),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization (by norm_num : Nat.Prime 5), Finsupp.
    single_eq_same]; norm_num [nsmul_eq_mul]

/-- Non-square witness n = 15: `ν_3(15! * 15#) = 7` (odd); `15! * 15# =
    3^7 * 17955857920000`, and `3 ∤ 17955857920000`. -/
theorem nonsquare_at_15 : ¬ IsSquare (15 ! * primorial 15) := by
  have hM : (15 : ℕ) ! * primorial 15 = 3 ^ 7 * 17955857920000 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 3)
    (mul_ne_zero (Nat.factorial_ne_zero 15) (primorial_ne_zero 15))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^7) ≠ 0) (by
    norm_num : (17955857920000:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 17955857920000),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 16: `ν_3(16! * 16#) = 7` (odd); `16! * 16# =
    3^7 * 2872937726720000`, and `3 ∤ 2872937726720000`. -/
theorem nonsquare_at_16 : ¬ IsSquare (16 ! * primorial 16) := by
```

```
  have hM : (16 : ℕ) ! * primorial 16 = 3 ^ 7 * 287293726720000 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 3)
    (mul_ne_zero (Nat.factorial_ne_zero 16) (primorial_ne_zero 16))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^7) ≠ 0) (by
    norm_num : (287293726720000:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 287293726720000),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 17: `ν_3(17! * 17#) = 7` (odd); `17! * 17# =
    3^7 * 83027887022080000`, and `3 ∤ 83027887022080000`. -/
theorem nonsquare_at_17 : ¬ IsSquare (17 ! * primorial 17) := by
  have hM : (17 : ℕ) ! * primorial 17 = 3 ^ 7 * 83027887022080000 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 3) (e := 3)
    (mul_ne_zero (Nat.factorial_ne_zero 17) (primorial_ne_zero 17))
  rw [hM, Nat.factorization_mul (by norm_num : ((3:ℕ)^7) ≠ 0) (by
    norm_num : (83027887022080000:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (3:ℕ) ∣ 83027887022080000),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_three, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 18: `ν_2(18! * 18#) = 17` (odd); `18! * 18# =
    2^17 * 24936491397943125`, and `2 ∤ 24936491397943125`. -/
theorem nonsquare_at_18 : ¬ IsSquare (18 ! * primorial 18) := by
  have hM : (18 : ℕ) ! * primorial 18 = 2 ^ 17 * 24936491397943125 := by
    decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 8)
    (mul_ne_zero (Nat.factorial_ne_zero 18) (primorial_ne_zero 18))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^17) ≠ 0) (by
    norm_num : (24936491397943125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 24936491397943125),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 19: `ν_2(19! * 19#) = 17` (odd); `19! * 19# =
    2^17 * 9002073394657468125`, and `2 ∤ 9002073394657468125`. -/
theorem nonsquare_at_19 : ¬ IsSquare (19 ! * primorial 19) := by
  have hM : (19 : ℕ) ! * primorial 19 = 2 ^ 17 * 9002073394657468125 :=
    by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 8)
    (mul_ne_zero (Nat.factorial_ne_zero 19) (primorial_ne_zero 19))
```

```
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^17) ≠ 0) (by
    norm_num : (9002073394657468125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 9002073394657468125),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 20: `ν_2(20! * 20#) = 19` (odd); `20! * 20# =
    2^19 * 45010366973287340625`, and `2 ∤ 45010366973287340625`. -/
theorem nonsquare_at_20 : ¬ IsSquare (20 ! * primorial 20) := by
  have hM : (20 : ℕ) ! * primorial 20 = 2 ^ 19 * 45010366973287340625 :=
    by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 9)
    (mul_ne_zero (Nat.factorial_ne_zero 20) (primorial_ne_zero 20))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^19) ≠ 0) (by
    norm_num : (45010366973287340625:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 45010366973287340625),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Non-square witness n = 21: `ν_2(21! * 21#) = 19` (odd); `21! * 21# =
    2^19 * 945217706439034153125`, and `2 ∤ 945217706439034153125`. -/
theorem nonsquare_at_21 : ¬ IsSquare (21 ! * primorial 21) := by
  have hM : (21 : ℕ) ! * primorial 21 = 2 ^ 19 * 945217706439034153125 :=
     by decide
  apply not_isSquare_of_odd_factorization (p := 2) (e := 9)
    (mul_ne_zero (Nat.factorial_ne_zero 21) (primorial_ne_zero 21))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^19) ≠ 0) (by
    norm_num : (945217706439034153125:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by decide :
    ¬ (2:ℕ) ∣ 945217706439034153125),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same];
    norm_num [nsmul_eq_mul]

/-- Assembled form (cited by the ⟹ direction of the main theorem): `n!
    * n#` is non-square at every `n` with `6 ≤ n ≤ 21`. -/
theorem nonsquare_six_to_twenty_one (n : ℕ) (h6 : 6 ≤ n) (h21 : n ≤ 21)
    :
    ¬ IsSquare (n ! * primorial n) := by
  interval_cases n
  · exact nonsquare_at_6
  · exact nonsquare_at_7
  · exact nonsquare_at_8
  · exact nonsquare_at_9
  · exact nonsquare_at_10
```

```
  · exact nonsquare_at_11
  · exact nonsquare_at_12
  · exact nonsquare_at_13
  · exact nonsquare_at_14
  · exact nonsquare_at_15
  · exact nonsquare_at_16
  · exact nonsquare_at_17
  · exact nonsquare_at_18
  · exact nonsquare_at_19
  · exact nonsquare_at_20
  · exact nonsquare_at_21

/-- **Lemma B (interval prime lemma)**: for every integer `n ≥ 22` there exists a prime `p` with `n/3 < p ≤ n/2`.
    (`/` is natural-number division: the interval reads `⌊n/3⌋ < p ≤ ⌊n/2⌋`; for prime existence this coincides
    with the real reading, since `p` is an integer: `n/3 < p ⟺ ⌊n/3⌋ < p` and `p ≤ n/2 ⟺ p ≤ ⌊n/2⌋`.) -/
theorem exists_prime_between_div_three_and_div_two (n : ℕ) (hn : 22 ≤ n) :
    ∃ p : ℕ, p.Prime ∧ n / 3 < p ∧ p ≤ n / 2 := by
  rcases lt_or_ge n 1000000 with h | h
  · exact NFactPrimorialSquareChain.lemmaB_upto_999999 n hn (by omega)
  · exact NFactPrimorialSquareChain.lemmaB_tail_1000000 n h

/-- **Main theorem (complete solution set, iff form)**:
    `n! * n#` is a perfect square ⟺ `n ≤ 5` (for `n : ℕ`).
    Over ℕ, `n ≤ 5 ⟺ n ∈ {0,1,2,3,4,5}`, so this is the main
    statement "the solutions are exactly n ∈ {0,...,5}". -/
theorem isSquare_factorial_mul_primorial_iff (n : ℕ) :
    IsSquare (n ! * primorial n) ↔ n ≤ 5 := by
  constructor
  · -- ⟹ direction: n! * n# a square ⟹ n ≤ 5 (contrapositive: for n ≥ 6, n! * n# is not a square)
    intro h
    by_contra hle
    have h6 : 6 ≤ n := by omega
    rcases lt_or_ge n 22 with hlt | h22
    · -- the 6 ≤ n ≤ 21 segment: direct contradiction with the pointwise non-square witnesses of the witness layer
      exact nonsquare_six_to_twenty_one n h6 (by omega) h
    · -- the 22 ≤ n segment: Lemma B yields a prime p ∈ (n/3, n/2]; the criterion gives Odd (ν_p(n!)), but Legendre's sum evaluates ν_p(n!) = 2
      obtain ⟨p, hp, hpl, hpu⟩ := exists_prime_between_div_three_and_div_two n h22
      have hodd : Odd ((n !).factorization p) :=
        (isSquare_iff_all_factorization_odd n (by omega)).mp h p hp (by omega)
```

```
      have hn3p : n < 3 * p := by omega
      have hp3 : 3 ≤ p := by omega
      have hpsq : n < p ^ 2 := by
        have h3pp : 3 * p ≤ p * p := Nat.mul_le_mul hp3 (le_refl p)
        rw [pow_two]
        exact lt_of_lt_of_le hn3p h3pp
      have hlog2 : Nat.log p n < 2 := Nat.log_lt_of_lt_pow (by omega)
    hpsq
      have hd1 : n / p = 2 := by
        have hp0 : 0 < p := hp.pos
        have hr : n - 2 * p < p := by omega
        rw [show n = n - 2 * p + p * 2 by omega, Nat.add_mul_div_left _ _
     hp0,
          Nat.div_eq_of_lt hr, zero_add]
      have hfact2 : (n !).factorization p = 2 := by
        have hIco : Finset.Ico 1 2 = ({1} : Finset ℕ) := by decide
        calc (n !).factorization p = ∑ i ∈ Finset.Ico 1 2, n / p ^ i :=
              Nat.factorization_factorial hp hlog2
          _ = ∑ i ∈ ({1} : Finset ℕ), n / p ^ i := by rw [hIco]
          _ = n / p ^ 1 := Finset.sum_singleton _ _
          _ = n / p := by rw [pow_one]
          _ = 2 := hd1
      rw [hfact2] at hodd
      obtain ⟨k, hk⟩ := hodd
      omega
  · -- ⟸ direction: for n ≤ 5, give the explicit square root pointwise
    (x = 1, 1, 2, 6, 12, 60)
    intro h
    obtain ⟨h0, h1, h2, h3, h4, h5⟩ := witnesses_zero_to_five
    interval_cases n
    · exact (isSquare_iff_exists_sq _).mpr ⟨1, h0.symm⟩
    · exact (isSquare_iff_exists_sq _).mpr ⟨1, h1.symm⟩
    · exact (isSquare_iff_exists_sq _).mpr ⟨2, h2.symm⟩
    · exact (isSquare_iff_exists_sq _).mpr ⟨6, h3.symm⟩
    · exact (isSquare_iff_exists_sq _).mpr ⟨12, h4.symm⟩
    · exact (isSquare_iff_exists_sq _).mpr ⟨60, h5.symm⟩

/-- **Main theorem (existence-witness form, delivered alongside the iff
    form)**:
    the Diophantine equation `x^2 = n! * n#` has a solution ⟺ `n ∈
    {0,1,2,3,4,5}`. -/
theorem exists_sq_factorial_mul_primorial_iff (n : ℕ) :
    (∃ x : ℕ, x ^ 2 = n ! * primorial n) ↔ n ∈ ({0, 1, 2, 3, 4, 5} :
    Finset ℕ) := by
  have hbridge : (∃ x : ℕ, x ^ 2 = n ! * primorial n) ↔ IsSquare (n ! *
    primorial n) := by
    rw [isSquare_iff_exists_sq]
    constructor
    · rintro ⟨x, hx⟩
```

```
      exact ⟨x, hx.symm⟩
    · rintro ⟨r, hr⟩
      exact ⟨r, hr.symm⟩
  rw [hbridge, isSquare_factorial_mul_primorial_iff]
  simp only [Finset.mem_insert, Finset.mem_singleton]
  constructor <;> intro h <;> omega

end NFactPrimorialSquare
```

4 Verification evidence

- Toolchain: Lean `v4.34.0` (`leanprover/lean4:v4.34.0`), mathlib4 `v4.34.0`, revision `5ed29652`; the exact toolchain pin ships as `proofs/lean-toolchain`.
- Statement integrity: the frozen snapshot `nfact-primorial-square.statement.txt` has sha256

  `0b13d08d6d915c2ebb25a070461a9d2a`
  `7d0364b12b3627c1b1efba5cc9f8ea48`,

  the statement source `nfact-primorial-square.lean` has sha256

  `638064201e96a3bdb42e75b00020c16e`
  `2fb27c2738c58fa3d4923cde2a451a8f`,

  and the final proof file `final-nfact-primorial-square.lean` (1094 lines, LF) has sha256

  `276cb2e698f6d201752eed3cb1104254`
  `d976f1a392ba3860a194eaa66986070a`.

  The final audit compared all eight statement headers of the proof file against the frozen snapshot under two whitespace conventions (eight of eight match) and re-checked the kernel goal states against the snapshot's `#check` record (eight of eight match). The listing excerpts in this note are byte-identical to the shipped proof files (mechanical record: `audit/listing-verify.txt`). The three `proofs/` sources are comment-cleaned editions (English docstrings): their code regions are byte-identical to the frozen files listed above (correspondence: `audit/comment-cleanup-diff.md`), and the post-cleanup kernel re-verification is recorded in `audit/kernel-rerun.txt`.

- Kernel check: strict compilation of the final file exits 0 with no `sorry` and no `admit` (whole-file scan including comments: zero hits; likewise zero hits for `axiom`, `unsafe`, `opaque`, `native_decide`, `implemented_by`). Three diagnostics remain, all linter/deprecation-class and non-rejectable under the audit rules applied: an elaborator exponentiation-threshold hint on `18^100000` (intentional; the kernel checks the proof term independently), a deprecated `push_neg` alias, and one unused-variable linter note. `#print axioms` output, verbatim, re-printed independently by the packaging track on 2026-10-06 (strict compile exit 0; the wrapped fourth entry is terminal formatting):

```
'NFactPrimorialSquare.isSquare_factorial_mul_primorial_iff' depends
     on axioms: [propext, Classical.choice, Quot.sound]
'NFactPrimorialSquare.exists_sq_factorial_mul_primorial_iff'
    depends on axioms: [propext, Classical.choice, Quot.sound]
'NFactPrimorialSquare.isSquare_iff_all_factorization_odd' depends
    on axioms: [propext, Classical.choice, Quot.sound]
'NFactPrimorialSquare.exists_prime_between_div_three_and_div_two'
    depends on axioms: [propext,
 Classical.choice,
 Quot.sound]
'NFactPrimorialSquare.witnesses_zero_to_five' depends on axioms: [
    propext, Classical.choice, Quot.sound]
'NFactPrimorialSquare.primorial_factorization_eq_one' depends on
    axioms: [propext, Classical.choice, Quot.sound]
'NFactPrimorialSquare.primorial_factorization_eq_zero' depends on
    axioms: [propext, Classical.choice, Quot.sound]
'NFactPrimorialSquare.factorization_factorial_mul_primorial'
    depends on axioms: [propext, Classical.choice, Quot.sound]
```

  Every entry is exactly the whitelist {propext, `Quot.sound`, `Classical.choice`}; `sorryAx` does not occur.

- Grading by the independent final audit (adjudication of 2026-10-06, four checks all passed): *complete proof.* A cleaned copy of the adjudication ships as `audit/gate3-final-adjudication.txt` (translated to English; machine-local path prefixes replaced by `<REPO-ROOT>`; verdict text and data unchanged — normalization disclosed in `audit/package-notes.md`); the well-definedness gate output ships verbatim as `audit/welldef-nfact-primorial-square.txt`.

- Audit chain: statement frozen by hash before proving; post-hoc symmetric-difference comparison (text-level and kernel goal-state level); final strict recompile; axiom re-print (this note's copy:

`audit/axioms.out`); novelty record `audit/c9-record.md`. The claim audit `claim-check.md` is added to `audit/` after the independent claim audit returns its adjudication and is a prerequisite for publication.

5 Novelty evidence

Adjudication carried over from the independent novelty review: **no occupying record found**; search-strength label **strong**. The closing dates of the search are 2026-10-05 (deep-dig layer) and 2026-10-06 (independent review layer, which maintained every load-bearing claim). The full query log, including every zero-hit query, ships in full as `audit/c9-record.md` (English cleaned edition; conclusions and query data unedited). A zero hit is not novelty; the strongest claim licensed here is that no record of this object was found in the channels queried.

The problem is on record as open in the source paper. Novaković writes (verbatim wording; mathematical notation retypeset): "We believe that finitely many solutions occur for $x^2 = n! \cdot n\#$ or $x^2 = n!! \cdot [1, \ldots, n]$, but at the moment we were not able to give an argument" [3]. The present note answers the first of these two cases completely; the second case is *not* claimed here.

channel	queries	result
literature (author self-report)	arXiv:2601.16757 body text	the case is on record as open (quotation above) [3]
arXiv metadata	primorial ∧ factorial ∧ perfect square; primorial ∧ factorial ∧ square; double factorial ∧ least common multiple; squarefree part ∧ factorial	zero hits each
arXiv metadata	Brocard ∧ primorial	one hit [5]: adjacent object (primorial-core Pell), does not characterize this equation
OpenAlex full text	brocard/primorial/square/factorial family; k-free Brocard family	nearest hits include [3, 4]; no same-form statement
OEIS keyword	"n! primorial square"; "factorial times primorial is a square"	hits exist; no same-form
OEIS keyword	"double factorial lcm square"; "Novakovic factorial"	zero hits
public Lean ecosystem (formal-conjectures tarball, content level)	`primorial`; `doubleFactorial`	occurrences in unrelated files only; no item in this domain
OEIS value columns (review layer)	x: 1,1,2,6,12,60; x^2: 1,4,36,144,3600	no registration of these objects
local mathlib (pinned v4.34.0), positive control	content scan with live control	channel live; no same-form item

Delimitations against adjacent records (wording obligations carried from the source task card): (i) OEIS A055204 [8] asks for the *minimal* a making $n!\cdot a$ a square; the present theorem asks the *specific* value $a = n\#$ and is not an instance of A055204. (ii) Erdős problem #399 [7] concerns $n! = x^k \pm y^k$ — a different equation from $x^2 = n!\cdot n\#$ — and is marked DISPROVED (2026-02-04, counterexample $10! = 48^4 - 36^4$); it is recorded here only as same-family adjacency, never as an open anchor. (iii) arXiv:2602.09687 [4] is recorded with the restricted wording that *its object of study does not contain* $n\#$. (iv) arXiv:2606.23485 [5] (§5.3.1, primorial-core Pell equation $X^2 - DZ^2 = 1$) is an adjacent object and does not characterize which n make $n!\cdot n\#$ a square. (v) arXiv:2610.01899 [6] (square products of factorials and a conjecture of Erdős–Graham) is a different object; no collision, but a citation duty is recorded and discharged here. (vi) OEIS A166968 [9] (whose $a(2) = 8$ says: for $m \geq 8$ there is a prime in $(m, 3m/2]$) provides a literature-level shortest path toward Lemma 4 in the m-parametrization; the kernel proof shipped here does not go through it. The failure set of the interval property below $n = 22$ is $\{2, 3, 9, 21\}$ under the domain convention $n \geq 2$ ($n = 0, 1$

also fail, on an empty interval; this is irrelevant to Lemma 4, which starts at $n = 22$).

6 Scope and limitations

Grade and boundary, stated as the audit states them. (a) The kernel layer proves exactly the eight frozen statements of Listing 1; the grade of each is *complete proof* (zero `sorry`, axioms within the whitelist, statement text unchanged from the frozen snapshot, strict compile exit 0). Nothing beyond is claimed. (b) The second equation of the same remark, $x^2 = n!! \cdot [1, \ldots, n]$, is *not* claimed: only an exclusion path for even n was obtained. (c) No OEIS registration is claimed or performed by this deposit. (d) The exclusion of $n \geq 22$ is a pure theorem (Lemma 4) and does not rely on the Berndt–Galway computational bound (10^9) on the Brocard side; that bound plays no role here. (e) The hypothesis $n \geq 1$ of Lemma 3 is not used by its proof (the criterion holds at $n = 0$ as well); the lemma is stated as frozen. (f) A list of background items recorded as undecided at the full-text level is enumerated in the source task card (which stays in the source repository and is not shipped); none of them is load-bearing for this result and none is cited here as a zero hit. (g) This note contains no literature review; it is a deposit record, not a survey paper.

Data availability

Lean sources, verification and novelty-search evidence are deposited on Zenodo (concept DOI `https://doi.org/10.5281/zenodo.23177003`; the version history is recorded in `metadata/README.md` of the deposit), with a public mirror of the deposit contents on GitHub at `https://github.com/Aurora0134/ai4math-claims`. The deposit contains the claim note (LaTeX source and PDF), the Lean sources (`proofs/`: statement snapshot, statement source, final proof file, `lean-toolchain`), and the audit and novelty-search records (`audit/`). Claim note under CC BY 4.0, code under MIT.

Use of generative AI

The formalization and proofs reported in this note were developed with AI assistance (large language model: `kimi-k3`) under human direction and review. All statements and proofs reported here were checked by the Lean 4 kernel (Lean v4.34.0, mathlib v4.34.0): every proof compiles with no `sorry`,

and `#print axioms` reports only subsets of {propext, Quot.sound, Classical.choice}. The author reviewed all content and takes full responsibility for it.

Author contributions

The author selected and adjudicated the topic, verified the semantics of the formal statements, signed off the verification gates, and decided to deposit.

References

[1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *Automated Deduction – CADE-28*, LNCS, pp. 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37

[2] The mathlib Community. The Lean mathematical library. *CPP 2020*, pp. 367–381, 2020. https://doi.org/10.1145/3372885.3373824

[3] S. Novaković. The Diophantine equation $P(x) = \prod_{i=1}^{r} H_{n_i}$. arXiv:2601.16757, 2026. https://arxiv.org/abs/2601.16757

[4] S. Novaković. On a generalization of the Brocard–Ramanujan Diophantine equation. arXiv:2602.09687, 2026. https://arxiv.org/abs/2602.09687

[5] L. V. Peixoto. The Brocard problem: structural invariants, p-adic density, and the generative sieve. arXiv:2606.23485, 2026. https://arxiv.org/abs/2606.23485

[6] F. Yudin. Square products of factorials and a conjecture of Erdős and Graham. arXiv:2610.01899, 2026. https://arxiv.org/abs/2610.01899

[7] Erdős Problems website. Problem #399 ($n! = x^k \pm y^k$; marked DISPROVED, counterexample $10! = 48^4 - 36^4$). https://www.erdosproblems.com/399

[8] OEIS Foundation. Squarefree part of $n!$: $n!$ divided by its largest square divisor (sequence A055204). *The On-Line Encyclopedia of Integer Sequences*. https://oeis.org/A055204

[9] OEIS Foundation. Minimum k such that for all $m \geq k$ there is a prime p with $m < p < m(n+1)/n$ (sequence A166968). *The On-Line Encyclopedia of Integer Sequences*. `https://oeis.org/A166968`